StyleSathi Backend

API Documentation & Frontend Integration Guide

FastAPI | Supabase | PostgreSQL / pgvector | Stripe Demo

Generated from StyleSathi Backend codebase

Table of Contents

1. Architecture Overview

StyleSathi is a virtual fashion try-on and clothing search backend. Users can speak or type a clothing query, get AI-ranked results from local curated products and affiliate APIs (Amazon, Flipkart, Daraz), then generate a photo-realistic try-on image using FLUX.1.

Tech Stack

* **FastAPI**
  Python web framework
* **Supabase (PostgreSQL)**
  Auth + Database + Storage
* **pgvector (VECTOR(1024))**
  Vector search
* **BGE-M3 (local) or OpenAI**
  Embeddings
* **faster-whisper (local) or OpenAI Whisper**
  Voice transcription
* **FLUX.1 via Replicate or Colab**
  Try-on generation
* **Stripe (demo mode fakes the flow)**
  Payments

Data Flow

Frontend (React Native / Web) -> REST API (FastAPI) -> Supabase

Search pipeline:

1. POST /search?query=...
2. BGE-M3 generates query embedding (1024-dim)
3. Vector search in products table (if location = NP)
4. Keyword extraction -> affiliate API search
5. Affiliate results ranked by cosine similarity
6. Returns top 3 (for try-on) + more results

Try-on pipeline:

1. POST /try-on/generate (user photo + product image)
2. Usage check (free tries or subscription daily limit)
3. FLUX.1 generates wearing image
4. Saved to Supabase Storage
5. Returns public image URL

2. Authentication Flow

StyleSathi uses Supabase Auth (JWT-based). The frontend must handle user sign-up/sign-in via Supabase directly, then pass the access token to the backend in the Authorization header.

Step-by-step

* **1**
  User signs up or logs in via the Supabase client on the frontend
* **2**
  Supabase returns a JWT access token
* **3**
  Frontend sends every API request with header:

```
Authorization: Bearer <JWT_TOKEN>
```

* **4**
  Backend validates the JWT with Supabase Auth
* **5**
  If valid, request proceeds; if not, 401 is returned

[!] The backend does NOT handle sign-up/login. Use the Supabase JS/Flutter SDK on the frontend for authentication.

Token structure

The JWT contains:
- sub: User ID (UUID, used as primary key in users table)
- email: User's email address
- user_metadata: contains full_name, avatar_url, etc.

3. API Endpoints Reference

Base URL: http://<host>:8000

3.1 User Endpoints

GET `/user/me`

Get the current authenticated user's profile. Creates user record if first time.

Auth: Required

Response:

```
{
  "id": "uuid",
  "email": "user@example.com",
  "full_name": "John Doe",
  "vibes": ["casual", "formal"],
  "language_preference": "en",
  "onboarding_completed": false,
  "subscription_status": "inactive",
  "free_tries_used": 0,
  "free_tries_limit": 3,
  "user_usage": 0,
  "daily_limit": 20
}
```

PATCH `/user/me/update`

Update user profile fields.

Auth: Required

Request body:

```
{
  "full_name": "New Name",
  "vibes": ["streetwear", "minimalist"],
  "language_preference": "ne",
  "onboarding_completed": true
}
```

Response:

```
{"id": "uuid", "email": "...", ...}
```

GET `/user/me/tries`

Get free try-on tries remaining.

Auth: Required

Response:

```
{
  "used": 1,
  "limit": 3,
  "remaining": 2
}
```

3.2 Audio / Voice

Transcribe voice input (Nepali/English) to text for search queries.

POST `/audio/speech-to-text`

Upload an audio file and get transcribed text. Supports WAV, MP3, M4A.

Auth: Required

Request body:

```
Form-data: file=<audio_file>
```

Response:

```
{"text": "show me a red kurta"}
```

[!] VOICE_SERVICE_TYPE=local uses faster-whisper (CPU). First call downloads ~1.5GB model.

3.3 Preferences

GET `/preferences/me`

Get the user's preferences JSON.

Auth: Required

Response:

```
{"preferences": {"style": "casual", "size": "M"}}
```

PATCH `/preferences/me`

Set or update preferences (stored as JSONB).

Auth: Required

Request body:

```
{"style": "formal", "size": "L", "color": "blue"}
```

Response:

```
{"success": true}
```

3.4 Subscription & Payments

Uses Stripe Checkout Sessions. When STRIPE_SECRET_KEY is empty or a placeholder, demo mode activates - no real payment needed. Demo sessions are prefixed with demo_.

GET `/subscription/plans`

List available subscription plans (public, no auth required).

Auth: None

Response:

```
{
  "plans": [
    {"id": "basic", "name": "Basic", "price": {"amount": 499, "currency": "USD"}, "days": 30},
    {"id": "standard", "name": "Standard", "price": {"amount": 999, "currency": "USD"}, "days": 30},
    {"id": "pro", "name": "Pro", "price": {"amount": 1999, "currency": "USD"}, "days": 30}
  ]
}
```

POST `/subscription/create`

Create a checkout session for a plan.

Auth: Required

Request body:

```
{"planId": "basic"}
```

Response:

```
{
  "session_id": "cs_test_...",
  "payment_url": "https://checkout.stripe.com/..."
}
```

In demo mode, payment_url is a fake URL and session_id starts with demo_.

POST `/subscription/verify`

Verify payment and activate subscription. Demo mode immediately activates.

Auth: Required

Request body:

```
{"session_id": "cs_test_xxx"}
```

Response:

```
"completed"
```

3.5 Search

The core AI pipeline. Accepts a text query, detects user location, searches vector DB and affiliate APIs, ranks by semantic similarity.

POST `/search/search?query=<text>`

Search for fashion products. Returns top 3 for try-on + more results.

Auth: Required

Request body:

```
Query params: query (required), location (optional: NP/IN/US), category (optional), limit (default 20)
```

Response:

```
{
  "query": "red kurta",
  "keywords": ["red", "kurta"],
  "location": "NP",
  "total_results": 5,
  "top_results": [
    {"title": "...", "price": 1500, "image_url": "...", "score": 0.92}
  ],
  "more_results": [...]
}
```

Location detection: reads language_preference from user profile.

ne/NP -> Nepal (vector DB + Daraz)

hi/IN -> India (Flipkart + Amazon.in)

other -> US (Amazon.com)

Affiliate APIs require keys in .env. If blank, they return empty results silently.

3.6 Try-On Generation

POST `/try-on/generate`

Generate a try-on image: user photo + product clothing image. Checks usage limits first.

Auth: Required

Request body:

```
Form-data:
  user_image: <file> (photo of user)
  product_image: <file> (clothing image)
  prompt: (optional) custom FLUX prompt
```

Response:

```
{
  "generated_image_url": "https://...supabase.co/...",
  "usage_remaining": 2
}
```

[!] Requires REPLICATE_API_TOKEN or COLAB_FLUX_URL in .env. Without these, the endpoint returns 500.

4. Frontend Integration Guide

4.1 Initialization (React Native example)

```
// 1. Install supabase-js
npm install @supabase/supabase-js

// 2. Initialize Supabase client
import { createClient } from '@supabase/supabase-js'

const supabase = createClient(
  'https://oqdpwtyjbgzufeblrnww.supabase.co',
  'sb_publishable_O_uy5g170K9G0w9zWBLCCQ_ZCp2o3x4'
)
```

4.2 Authentication (Sign-up / Login)

```
// Sign Up
const { data, error } = await supabase.auth.signUp({
  email: 'user@example.com',
  password: 'secure_password',
  options: { data: { full_name: 'John Doe' } }
})

// Sign In
const { data, error } = await supabase.auth.signInWithPassword({
  email: 'user@example.com',
  password: 'secure_password'
})

// Get token
const token = data.session.access_token
```

4.3 Calling the Backend API

```
const API_BASE = 'http://your-backend:8000'

async function apiCall(method, path, body?, params?) {
  const token = (await supabase.auth.getSession())
    .data.session?.access_token

  const url = new URL(API_BASE + path)
  if (params) url.search = new URLSearchParams(params)

  const res = await fetch(url, {
    method,
    headers: {
      'Authorization': `Bearer ${token}`,
      'Content-Type': 'application/json',
    },
    body: body ? JSON.stringify(body) : undefined,
  })
  if (!res.ok) throw await res.json()
  return res.json()
}
```

4.4 Search Flow (Voice or Text)

```
// 1. User speaks -> transcribe
const formData = new FormData()
formData.append('file', audioBlob, 'audio.wav')
const { text } = await fetch(
  `${API_BASE}/audio/speech-to-text`,
  { method: 'POST', headers: { 'Authorization': `Bearer ${token}` },
    body: formData }
).then(r => r.json())

// 2. Search products
const results = await apiCall(
  'POST', '/search/search', null,
  { query: text, location: 'NP' }
)

// 3. top_results are the top 3 -> show with 'Try On' button
// more_results are the rest -> show in a scrollable list
```

4.5 Try-On Flow

```
const formData = new FormData()
formData.append('user_image', userPhoto, 'user.jpg')
formData.append('product_image', productImg, 'product.jpg')

const res = await fetch(`${API_BASE}/try-on/generate`, {
  method: 'POST',
  headers: { 'Authorization': `Bearer ${token}` },
  body: formData,
})
const { generated_image_url } = await res.json()
// Display the result image in an <Image> component
```

4.6 Subscription Flow (Demo Mode)

```
// 1. Get plans
const plans = await apiCall('GET', '/subscription/plans')

// 2. Create checkout
const { session_id, payment_url } = await apiCall(
  'POST', '/subscription/create',
  { planId: 'basic' }
)

// 3. In demo mode, just call verify directly
const status = await apiCall(
  'POST', '/subscription/verify',
  { session_id }
)
// status = 'completed' -> subscription activated

// 4. Redirect to payment_url in real Stripe mode
// Stripe redirects back to success_url with session_id
// Then call verify on return
```

4.7 Checking User State

```
// On app launch, check user state
const user = await apiCall('GET', '/user/me')

// Check tries remaining
const tries = await apiCall('GET', '/user/me/tries')

// Check if onboarding is needed
if (!user.onboarding_completed) {
  // Show vibe/language picker screen
  await apiCall('PATCH', '/user/me/update', {
    vibes: ['casual', 'formal'],
    language_preference: 'en',
    onboarding_completed: true
  })
}
```

5. Setup Checklist

Required steps:

[ ] Clone the repo and cd into Backend/

[ ] Create .env with your Supabase credentials (already done)

[ ] Run supabase_migration.sql in Supabase SQL Editor

[ ] pip install -r requirements.txt

[ ] python scripts/seed_products.py

[ ] python scripts/seed_embeddings.py

[ ] Create Supabase Storage bucket 'tryon-images' (public)

[ ] uvicorn app:app --reload

Optional / when needed:

[ ] Set STRIPE_SECRET_KEY for real payment processing

[ ] Set REPLICATE_API_TOKEN to enable FLUX try-on

[ ] Set COLAB_FLUX_URL as alternative to Replicate

[ ] Add Amazon, Flipkart, Daraz affiliate keys for live product search

[ ] Change VOICE_SERVICE_TYPE=cloud and set OPENAI_API_KEY for cloud whisper

[ ] Change EMBEDDING_PROVIDER=openai for cloud embeddings

6. Environment Variables Reference

SUPABASE_PROJECT_URL	Your Supabase project URL
SUPABASE_API_KEY	Supabase anon/publishable key
SUPABASE_SERVICE_KEY	Supabase service_role key (bypasses RLS)
OPENAI_API_KEY	Used only if VOICE_SERVICE_TYPE=cloud or EMBEDDING_PROVIDER=openai
VOICE_SERVICE_TYPE	'local' (faster-whisper) or 'cloud' (OpenAI)
STRIPE_SECRET_KEY	Stripe secret key. Empty/placeholder = demo mode
EMBEDDING_PROVIDER	'local' (BGE-M3) or 'openai'
EMBEDDING_DIMENSION	Embedding vector dimension (default 1024)
REPLICATE_API_TOKEN	Replicate API token for FLUX try-on
COLAB_FLUX_URL	Colab-hosted FLUX endpoint URL
STORAGE_BUCKET	Supabase Storage bucket for try-on images
AMAZON_ACCESS_KEY	Amazon PAAPI access key
AMAZON_SECRET_KEY	Amazon PAAPI secret key
AMAZON_ASSOCIATE_TAG	Amazon affiliate tag
FLIPKART_AFFILIATE_ID	Flipkart affiliate ID
FLIPKART_API_KEY	Flipkart affiliate API key
DARAZ_API_KEY	Daraz API key